

MPCGPU CUDA → HIP Porting Plan

goal: run
examples/track_iiwa_pcg.cu

1. What are we trying to port?

Define the first target narrowly so we can actually finish it.

Primary goal

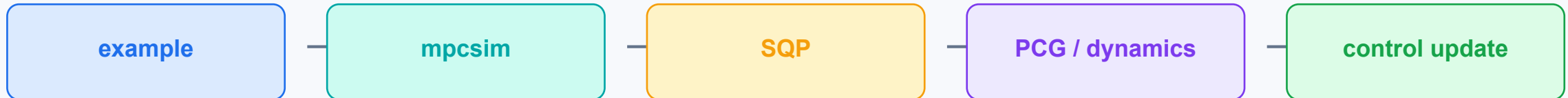
- Port enough CUDA/HIP code to run examples/track_iiwa_pcg.cu.
- PCG path is the meaningful MPCGPU solver path.
- QDLDL path is useful later, but not first priority (just we have 2 examples but we focus on PCG)

first questions

- we should ask supervisor if QDLDL is actually in scope of our project (i doubt)

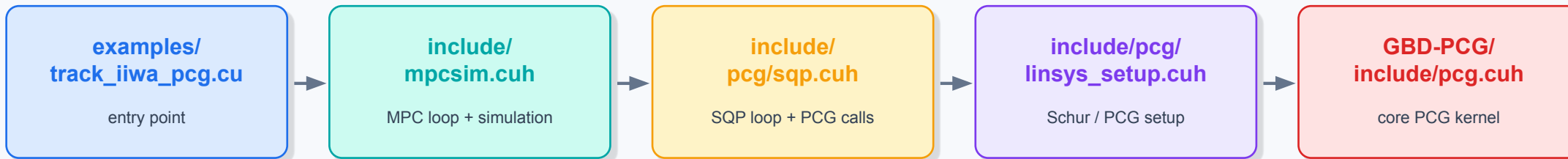
Out of first scope

- Do not port the GRiD generator itself yet.
- Do not start with the QDLDL path.
- Do not chase performance until correctness works.



2. Repository map: where the main PCG example goes

This is the path we should keep in our heads when dividing work.



Side components called by the path

- include/common/: KKT, merit, integrator, dz, settings ->math
- include/dynamics/iiwa/: generated robot dynamics kernels ->grid
- GLASS + GBD-PCG/GLASS: GPU math helpers used by PCG

Why this matters

- If a file is not on this path, it is not needed for the first port target.
- If a CUDA feature appears on this path, it needs an isolated test.
- Later QDLDL path is separate: examples/track_iiwa_qdldl.cu.

3. What counts as “CUDA to HIP work”?

Not only API calls: CUDA language features and project-specific kernels can break too.

API calls

- Runtime: `cudaMalloc`, `cudaMemcpy`, `cudaFree`, `cudaMemset`.
- Streams: `cudaStreamCreate`, `cudaMemcpyAsync`.
- Device/launch: `cudaGetDeviceProperties`, `cudaLaunchCooperativeKernel`.
- Libraries: `cublasSaxpy`, `cublasDaxpy`.

CUDA language features

- `__global__`, `__device__`, `__host__`.
- `threadIdx`, `blockIdx`, `blockDim`, `gridDim`.
- `__shared__`, `extern __shared__`, `__syncthreads()`.
- `atomicAdd` and dynamic shared memory.

Project-specific GPU code

- PCG kernel and solvePCG interface.
- GLASS: dot, reduce, gemv, gemm, inverse, Cholesky.
- Generated GRiD dynamics kernels.
- KKT, merit, integrator, dz kernels.

Rule: if it influences compilation, launch behavior, synchronization, or numerical output, it needs either an isolated test or a component test (integrated with other components to work).

4. Work split: divide by architecture, not random files

so, we will divide the local tests to rprepare us for our parts as well. this is how we can work independently and in our tempo

Masha: core solver path

- GBD-PCG/include/*: PCG kernel, interface, constants, utility code.
- include/pcg/*: SQP using PCG, linear-system setup.
- GLASS and GBD-PCG/GLASS: GPU math helpers.
- High-risk features: cooperative launch, grid sync, cuBLAS/hipBLAS, occupancy.

•
PCG solver, preconditioner, Schur/linear-system setup, GLASS, cooperative kernels

main deliverable: PCG component can compile + run tiny tests

Tobias: outer MPC + dynamics path

- examples/track_iiwa_pcg.cu and include/mpcsim.cuh.
- include/common/*: KKT, merit, integrator, dz, settings.
- include/dynamics/*: generated GRiD robot dynamics code.
- High-risk features: streams, async copy, dynamic shared memory attributes, atomics.

•
NMPC wrapper, example flow, dynamics, integrator, KKT matrix generation, merit/tracking/error kernels

main deliverable: outer kernels + generated dynamics compile/run

5. Masha scope: core solver and risky synchronization

This is the part closest to the paper's PCG contribution.

Area	Files	Purpose
GBD-PCG solver	GBD-PCG/include/pcg.cuh GBD-PCG/include/interface.cuh GBD-PCG/include/gpu_pcg.cuh	Core PCG loop, cooperative kernel, public solve interface
GBD-PCG support	types.cuh, constants.cuh utils.cuh, gpuassert.cuh	Types, launch constants, reductions, error macros
MPCGPU PCG path	include/pcg/sqp.cuh include/pcg/linsys_setup.cuh	SQP-PCG integration, Schur / Sy / Pinv setup
GLASS math	GLASS/glass.cuh GLASS/src/L1,L2,L3/* GBD-PCG/GLASS/*	Device math: copy, dot, reduce, gemv, gemm, inverse, Cholesky

Masha’s question after isolated tests: can the PCG kernel and GLASS helpers compile and execute under HIP?

6. Tobias scope: outer MPC, common kernels, generated dynamics

This is the path around the solver: input/output, dynamics, integration, costs.

Area	Files	Purpose
Example + loop	examples/track_iiwa_pcg.cu include/mpcsim.cuh	Main entry point, simulation wrapper, trajectory shifting
Common kernels	include/common/dz.cuh integrator.cuh, kkt.cuh merit.cuh, settings.cuh	Trajectory correction, integration, KKT pieces, merit/line-search
Generated dynamics	include/dynamics/rbd_plant.cuh include/dynamics/iiwa/*_grid.cuh include/dynamics/iiwa/*_plant.cuh	GRiD-generated robot dynamics and derivatives
Utilities	include/utls/csr.cuh include/utls/matrix.cuh include/utls/experiment.cuh	Sparse/matrix helpers and experiment output

Tobias’s question after isolated tests: can the outer MPC/dynamics kernels compile and run under HIP?

7. Masha feature groups to test first

These are the highest-risk core solver features.

Group	Feature / API	Why it matters
A	cudaMalloc / cudaMemcpy / cudaFree	Basic memory flow; already started.
B	cudaDeviceSynchronize / cudaPeekAtLastError	Error handling and correctness checkpoints.
C	cudaLaunchCooperativeKernel	Required for grid-wide sync in PCG.
D	cooperative_groups::this_grid() + grid.sync()	Biggest risk: full-grid synchronization.
E	extern __shared__ + dynamic shared memory	PCG and Schur kernels depend on it.
F	cudaOccupancyMaxActiveBlocksPerMultiprocessor	Used to compute PCG launch configuration.
G	cudaGetDeviceProperties / cudaDeviceGetAttribute	Needed for device capability checks.
H	cuBLAS AXPY	Library migration: cuBLAS → hipBLAS/rocBLAS.
I	GLASS copy/dot/reduce/gemv/gemm/inv	Not API calls, but core solver math.

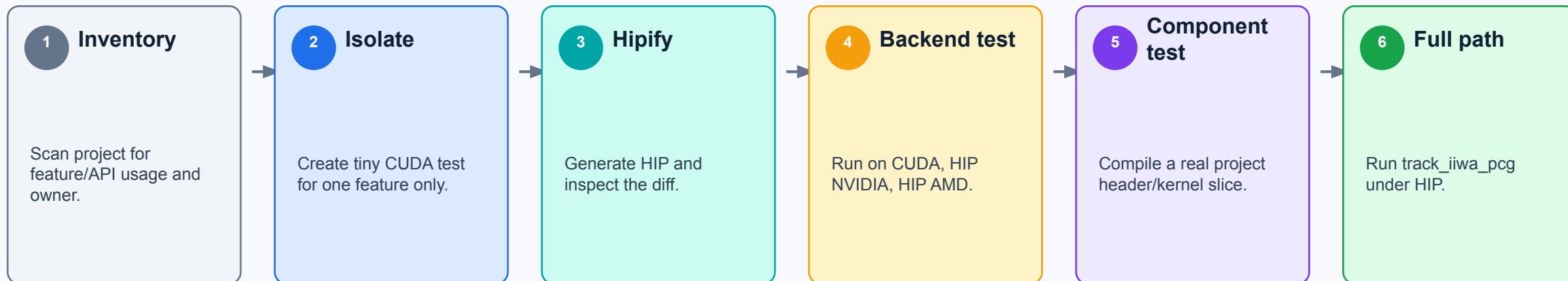
8. Tobias feature groups to test first

These cover normal kernels, generated dynamics, streams, and common MPC utilities.

Group	Feature / API	Why it matters
J	normal kernel<<<blocks,threads,smem>>> launch	Used by dynamics, KKT, merit, integrator, dz.
K	threadIdx / blockIdx / blockDim / gridDim	Massive use in GRiD-generated code.
L	__global__ / __device__ / __host__	Core CUDA language annotations.
M	__shared__ / extern __shared__ / __syncthreads()	Common in dynamics and integration kernels.
N	cudaMemcpyAsync	Used by generated dynamics and async data movement.
O	cudaStream_t + stream priority	Generated dynamics uses stream management.
P	cudaFuncSetAttribute + MaxDynamicSharedMemorySize	Needed for large dynamic shared memory kernels.
Q	cudaMemset	Used in simulation/common path.
R	cudaMemcpy2D	Used by matrix utility.
S	atomicAdd	Tracking error and merit accumulation.
T	generated GRiD kernel compile/run	Proves generated CUDA output survives HIP.

9. Testing ladder: how we avoid drowning in full-repo errors

Every feature should climb the same ladder from tiny test to real repository integration.



Rule of thumb

- Do not debug a full repository error until the same feature works in isolation.
- Do not optimize until correctness passes on both backends.
- Do not assume HIPIFY means correctness; compile + run decides.

10. Isolated tests for Masha

These tests are small programs in the sandbox before touching real PCG code.

ID	Test file	Validates
001	001_basic_memory.cu	cudaMalloc / cudaMemcpy / cudaFree
002	002_error_checking.cu	error macros, cudaPeekAtLastError, cudaDeviceSynchronize
003	003_dynamic_shared_memory.cu	extern __shared__, smem launch parameter
004	004_cooperative_groups_grid_sync.cu	cooperative_groups::this_grid(), grid.sync()
005	005_cooperative_kernel_launch.cu	cudaLaunchCooperativeKernel → hipLaunchCooperativeKernel
006	006_occupancy_and_device_props.cu	device properties and occupancy calculation
007	007_cublas_axpy.cu	cuBLAS AXPY → hipBLAS path
008	008_glass_copy_reduce_dot.cu	GLASS L1 operations
009	009_glass_gemv_gemm.cu	GLASS L2/L3 operations
010	010_small_pcg_like_kernel.cu	tiny PCG-like loop with sync + reductions

11. Isolated tests for Tobias

These tests cover the outer path and generated-code patterns.

ID	Test file	Validates
011	011_basic_kernel_launch.cu	normal kernel launch
012	012_thread_indexing_grid_stride.cu	thread/block/grid indexing and grid-stride loops
013	013_static_and_dynamic_shared_memory.cu	static + dynamic shared memory
014	014_async_copy_and_stream.cu	cudaMemcpyAsync and stream use
015	015_stream_priority.cu	stream priority creation/destruction
016	016_func_set_attribute_dynamic_smem.cu	cudaFuncSetAttribute for dynamic smem
017	017_atomic_add.cu	atomicAdd correctness
018	018_cuda_memset.cu	cudaMemset → hipMemset
019	019_cuda_memcpy2d.cu	cudaMemcpy2D → hipMemcpy2D
020	020_grid_generated_kernel_compile_test.cu	small extracted GRiD-style kernel
021	021_integrator_kernel_compile_test.cu	compile/run integrator-like kernel
022	022_merit_or_kkt_kernel_compile_test.cu	compile/run merit/KKT-like kernel

12. After isolated tests: how Masha tests real repository code

Core solver components can be tested without running the full MPC loop first.

Component test strategy

- Create a tiny "pcg_component_test" that includes GBD-PCG headers only.
- Allocate small synthetic SPD/block-like system data.
- Launch PCG or a stripped PCG-like cooperative kernel.
- Verify residual decreases or known toy solution is recovered.
- Then include include/pcg/linsys_setup.cuh and test Schur setup separately.

Why not full example immediately?

- Full example depends on dynamics, KKT, merit, integrator, CSV inputs, and PCG.
- If it fails, we will not know which layer failed.
- Core PCG should become a standalone confidence checkpoint first.

Suggested real-repo tests

- T1: compile GBD-PCG/include/pcg.cuh with HIP.
- T2: run a minimal cooperative PCG launch.
- T3: compile GLASS helper headers under HIP.
- T4: link PCG + GLASS + linsys_setup in tiny executable.
- T5: integrate into track_iiwa_pcg after Tobias side compiles.

13. After isolated tests: how Tobias tests real repository code

Outer-loop components can be tested as smaller kernels before the full example.

Component test strategy

- Start from `examples/track_iiwa_pcg.cu`, but do not run the full path immediately.
- Compile generated dynamics headers and call one small dynamics kernel.
- Compile/run integrator-like and merit/KKT-like kernels with dummy data.
- Check that streams, memcpy async, func attributes, atomics work under HIP.
- Only then connect the full mpcsim path.

Suggested real-repo tests

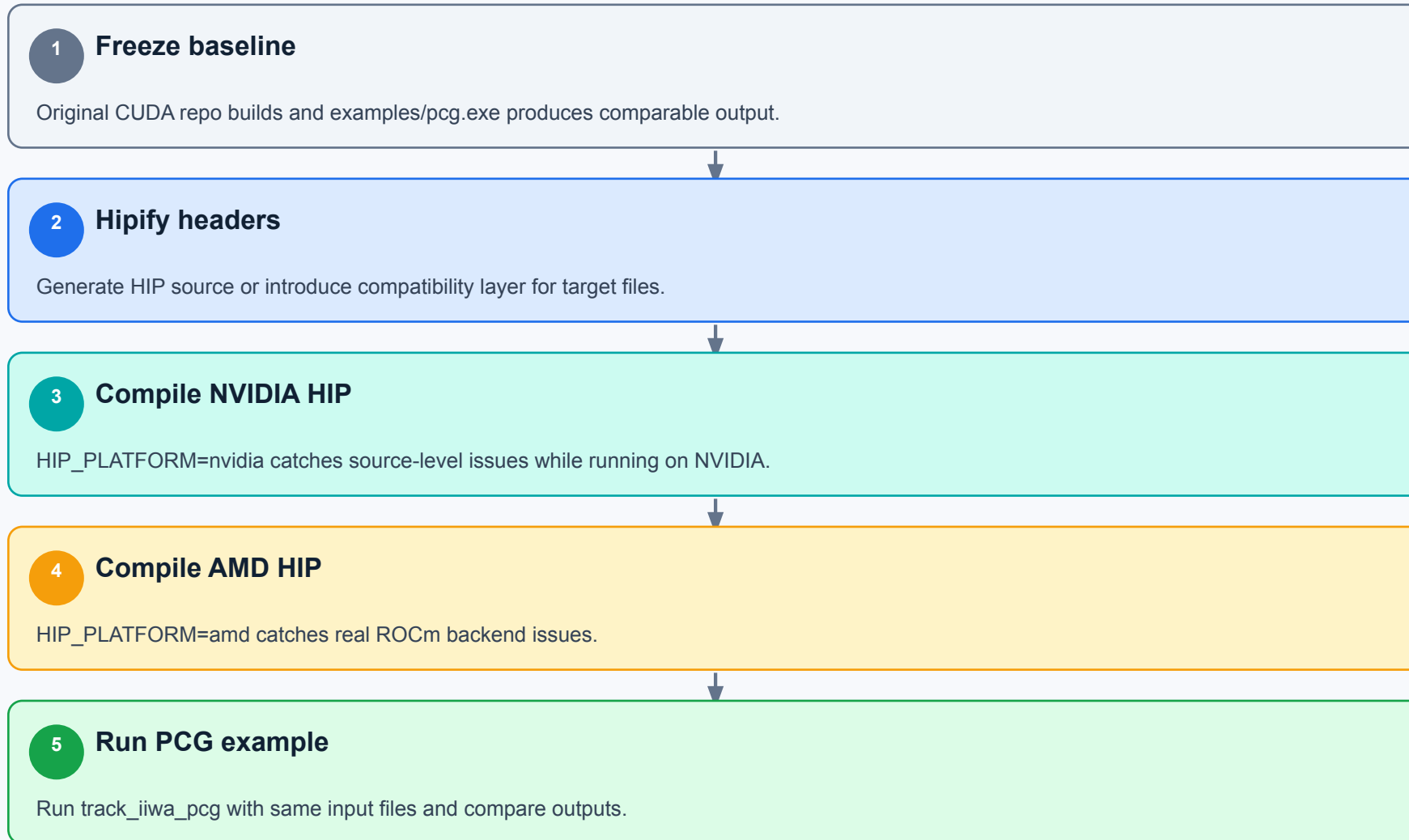
- T1: compile `include/dynamics/iiwa/iiwa_grid.cuh` under HIP.
- T2: run `end_effector_positions_kernel` or a smaller dynamics kernel.
- T3: compile `include/common/integrator.cuh` and run small integration test.
- T4: compile merit/KKT/dz kernels with dummy inputs.
- T5: plug into full `track_iiwa_pcg` once core PCG side is ready.

GRiD decision

- Do not port the generator first.
- Port/test the generated CUDA files already present in `include/dynamics/iiwa/`.
- Revisit generator only if generated files are impossible to maintain manually.

14. Full repository integration order

When isolated and component tests pass, we integrate in a controlled sequence.



Integration should be commit-by-commit, not one massive HIPIFY commit.

15. Test matrix: every important test should pass here

This keeps us honest: source portability is not enough; compile/run decides.

Test level	CUDA native	HIP NVIDIA backend	HIP AMD backend
Isolated API/feature test	make run-cuda	make run-hip-nvidia	make run-hip-amd
Core PCG component	CUDA component executable	HIP/NVIDIA component executable	HIP/AMD component executable
Outer dynamics component	CUDA component executable	HIP/NVIDIA component executable	HIP/AMD component executable
Full PCG example	examples/pcg.exe	HIP/NVIDIA track_iiwa_pcg	HIP/AMD track_iiwa_pcg
Correctness check	expected output / CSV	matches CUDA enough	matches CUDA enough

Backend meaning

- HIP NVIDIA backend: uses hipcc + nvcc + CUDA runtime.
- HIP AMD backend: uses hipcc + clang + ROCm/ROCDXG.
- Same HIP source; different compiled binaries.

Output rule

- Small tests: exact expected value.
- Numerical solver: tolerance-based comparison.
- Performance comes after correctness.

16. Small definitions for the risky CUDA/HIP features

These are the terms most likely to appear during debugging.

Term	Plain meaning	Why it matters here
Cooperative kernel	A kernel launched so all blocks can cooperate.	Needed for grid-wide synchronization in PCG.
grid.sync()	Synchronization across all blocks in a cooperative kernel.	If unsupported/broken, PCG logic can fail.
Dynamic shared memory	Shared memory size chosen at launch time.	PCG, dynamics, KKT-like kernels use variable shared buffers.
Occupancy	How many blocks can run per SM/CU.	Used to choose legal cooperative launch dimensions.
Stream	A queue of GPU operations.	Async copies and generated dynamics scheduling use streams.
hipBLAS/rocBLAS	HIP/ROCm replacement path for cuBLAS.	cuBLAS calls are not pure text replacement if linking changes.

17. Small definitions for the MPCGPU algorithm terms

Not every detail is needed for porting, but these words explain the file structure.

Term	Friendly definition
MPC	Controller repeatedly replans a short future trajectory and applies only the first action.
SQP	Method that solves nonlinear optimization through repeated quadratic subproblems.
KKT system	Linear system from the optimality conditions of a constrained QP.
Schur complement	Reduced system $S\lambda = \gamma$ solved instead of the full KKT system.
PCG	Iterative solver for symmetric positive definite linear systems; core solver here.
GRiD	Code generator; in this repo we port the generated dynamics files, not the generator first.
GLASS	GPU linear algebra helper code used by PCG: reduce, dot, gemv, gemm, inverse.
QDLDL	Alternative sparse solver path; secondary after the PCG path works.

18. Decisions to make together

These are the discussion points that should become shared team agreements.

Scope decisions

- Confirm first target: `examples/track_iiwa_pcg.cu` only.
- Confirm QDLDL is secondary/later.
- Confirm generated GRiD output is in scope, not GRiD generator yet.

Ownership decisions

- Masha owns `PCG/GBD-PCG/GLASS/include/pcg`.
- Tobias owns `example/mpcsim/common/dynamics/utils`.
- Both keep test naming and reporting consistent.

Technical decisions

- How to compare numerical outputs. (definetely later)
- Whether to keep both NVIDIA and AMD HIP build targets in repo (could be just NVIDIA but it hink both are fine as well)

Process decisions

- One folder for isolated tests with numbered files.
- One API/feature table updated after each passing test.
- Full repo integration starts only after risky features pass.

19. Practical timeline

This is realistic even if Tobias is away during Pentecost week.

Period	Masha	Tobias	Output
Now	Finalize plan and test harness	Agree on split	Shared scope + numbered tests
Pentecost week	Core feature tests A–I; PCG component experiments	Away / no blocking work	Risky core features understood
After vacation	Help integrate PCG path	Outer tests J–T + dynamics compile tests	Both halves ready for repo integration
Integration week	HIPify PCG/GLASS/include/pcg	HIPify example/common/dynamics	First HIP/NVIDIA full compile
Next stage	AMD backend fixes	AMD backend fixes	First HIP/AMD full run

Closing: the strategy in one picture

If we follow this, the port becomes a sequence of controlled experiments.



**The first success is not “the whole repo is ported.”
The first success is that every risky CUDA feature has a tiny passing
HIP test.**

Then full-repository porting becomes engineering, not guessing.